

Projet d'Apprentissage statistique

Rendu de projet : Modèle de classification des raisins

APM_4STA3_TA

Haobin JIANG et Lucien LE GALL

21 mars 2025



INSTITUT
POLYTECHNIQUE
DE PARIS

Sommaire

1	Partie I	2
1.1	Analyse uni- et bi-variée	2
1.2	Déroulement de l'ACP	4
1.3	Classification hiérarchique ascendante	5
2	Partie II : Choisir un modèle	7
2.1	La modélisation	8
2.2	ROC	9
2.3	Les erreurs et choix finale	11
3	Partie III	12
3.1	LDA	12
3.2	Courbe ROC pour LDA	13

1 Partie I

1.1 Analyse uni- et bi-variée

- Analyse univariée. On réalise des boxplots du jeu de données, en fonction du type de raisin (Besni ou Kecimen). Pour la variable **Eccentricity** le résultat fut le suivant :

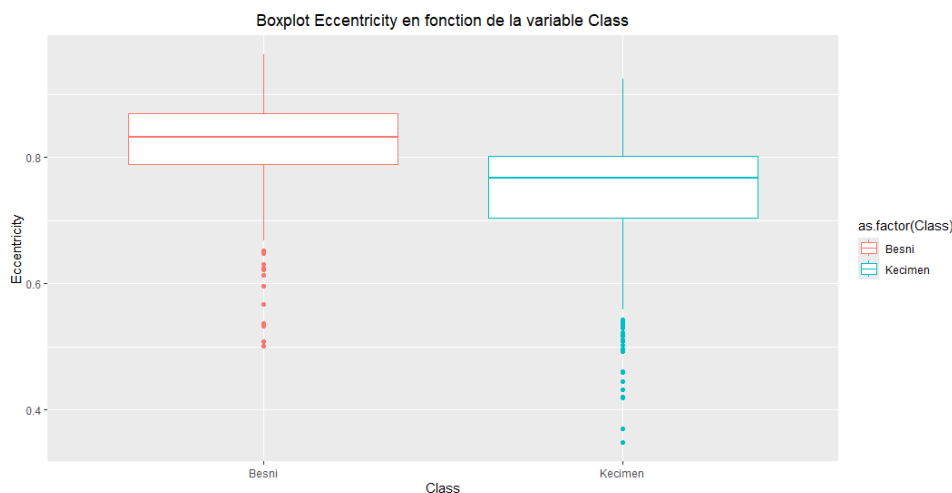


FIGURE 1 – Boxplot de la variable **Eccentricity** en fonction du type de raisin.

On peut d’ores et déjà constater qu’en moyenne les raisins de type Besni sont plus excentrés que ceux Kecimen. Les données sont plus resserrées pour Besni. Kecimen présente plus d’outliers. Cela suggère donc une certaine homogénéité pour les raisins de type Besni contrairement à l’hétérogénéité de Kecimen.

- Pour l’analyse bivariée, nous avons tout d’abord utilisé la fonction **corrplot**, dont les résultats donnent :

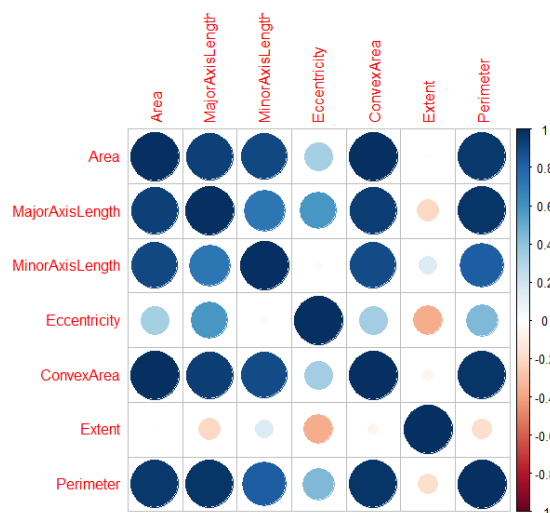


FIGURE 2 – Résultat de la fonction **corrplot**

On constate que certaines variables telles que **ConvexArea** et **Area** sont très fortement corrélées. *A contrario*, d’autres variables telles que **Eccentricity** et **Area**

sont décorrélées.

Enfin, une différenciation sur les nuages de points peut être faite suivant la classe. C'est particulièrement visible sur les variables décorrélées, telles que **Eccentricity** et **Area** :

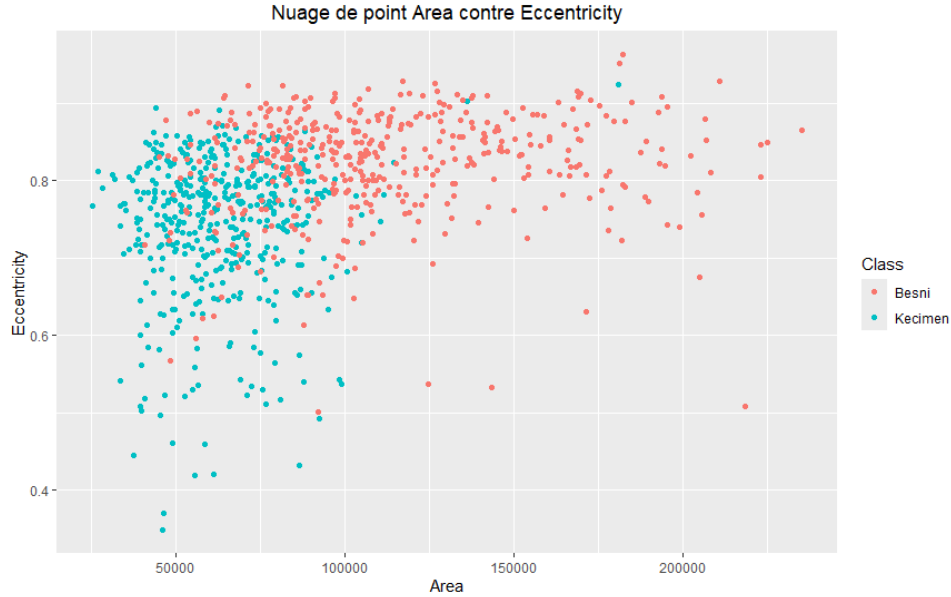


FIGURE 3 – Nuage de points **Area** contre **Eccentricity**

Ceci peut être fait pour toutes les variables, à l'aide de la fonction **ggpairs** :

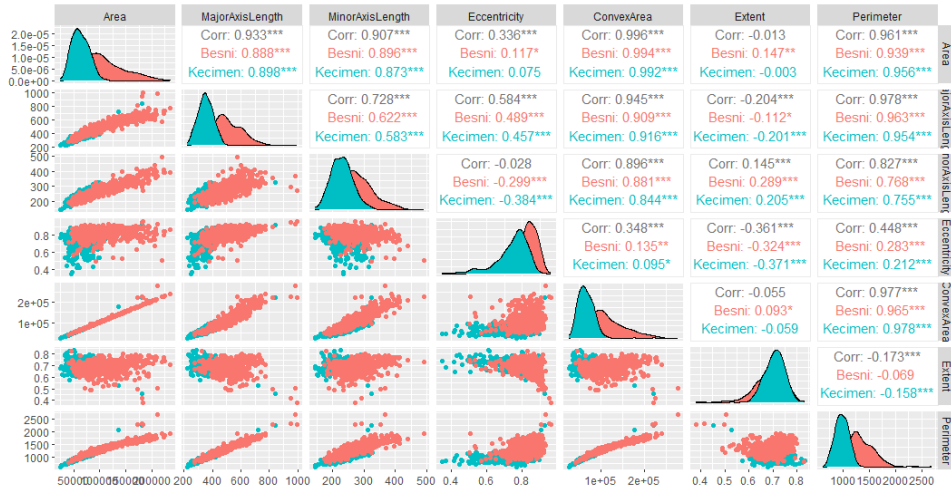


FIGURE 4 – Généralisation avec la commande **ggpairs**

On peut d'ores et déjà constater que l'étude du nuage de points est d'autant plus intéressante que les variables sont décorrélées, et que certaines variables telles que **Eccentricity** contrairement à **Extent** sont plus discriminantes pour les deux classes de raisins Besni et Kecimen.

Fort de ces observations, nous pouvons entamer l'analyse en composantes principales (ACP).

1.2 Déroulement de l'ACP

- Étape 0 : retirer la variable qualitative **Class** de l'étude.
- Étape 1 : Centrage et réduction des données. La réduction est ici essentielle car toutes les données ne possèdent pas la même unité.
- Étape 2 : Calcul des valeurs propres et ébouli des valeurs propres. On obtient comme résultat :

On constate qu'en conservant les deux premières directions propres, la somme de valeurs propres associées dépasse 80% de l'inertie totale. On fait donc le choix de conserver les deux premières directions propres. Ceci est conforté par l'application du critère du coude, sur l'ébouli des valeurs propres obtenu avec la commande **PCA** et **fviz_eig** :

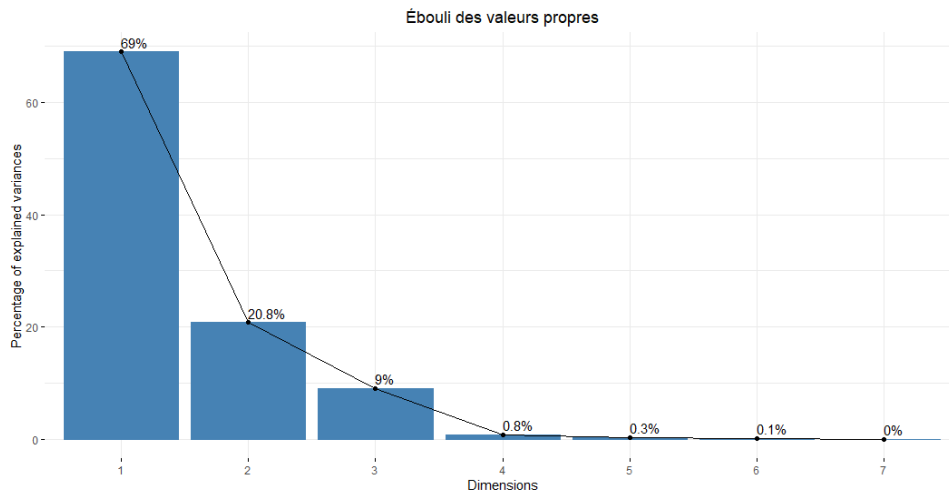


FIGURE 5 – Ébouli des valeurs propres de l'ACP normée, cassure dès la deuxième valeur propre

- Étape 3 : Tracé du cercle des corrélations et commentaires.

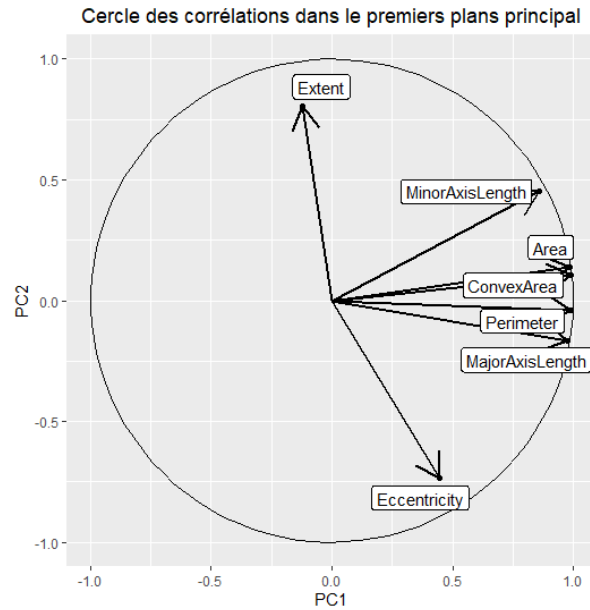


FIGURE 6 – Cercle des corrélations sur les deux premières direction propres après ACP normée

On constate un effet taille sur l'axe $G1$. Par ailleurs, les variables sont bien représentées, car proches du cercle des corrélations.

Enfin, la variable **Area** (et aussi **ConvexArea**) est décorrélée de **Extent**. Il en va de même pour la variable **Eccentricity** qui est décorrélée de **MinorAxisLength**.

1.3 Classification hiérarchique ascendante

Pour déterminer le nombre de clusters à conserver, on étudie l'indice de silhouette de plusieurs clustering avec différents niveaux de coupe dans le dendrogramme et on réalise des plots alluviaux. Avec et sans normalisation, les résultats sont les suivants :

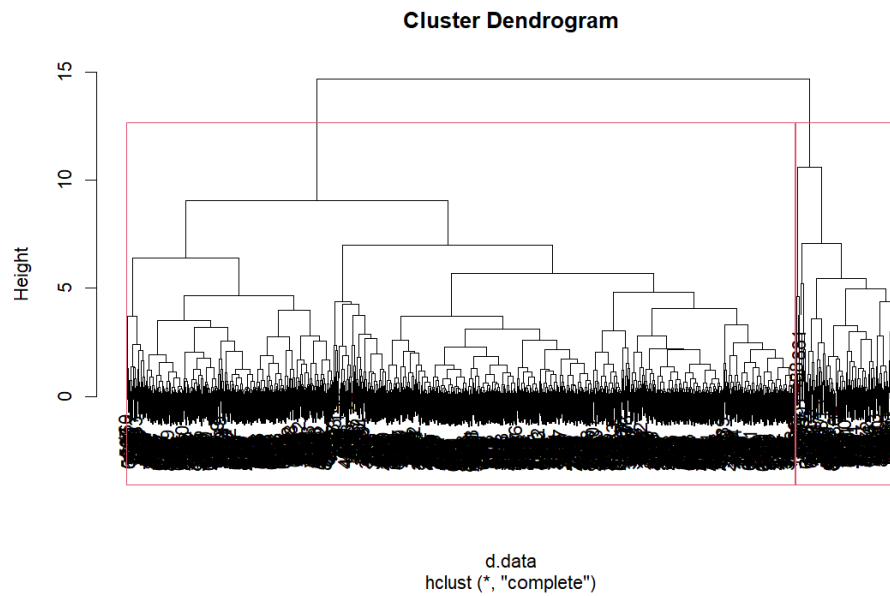


FIGURE 7 – Dendrogramme

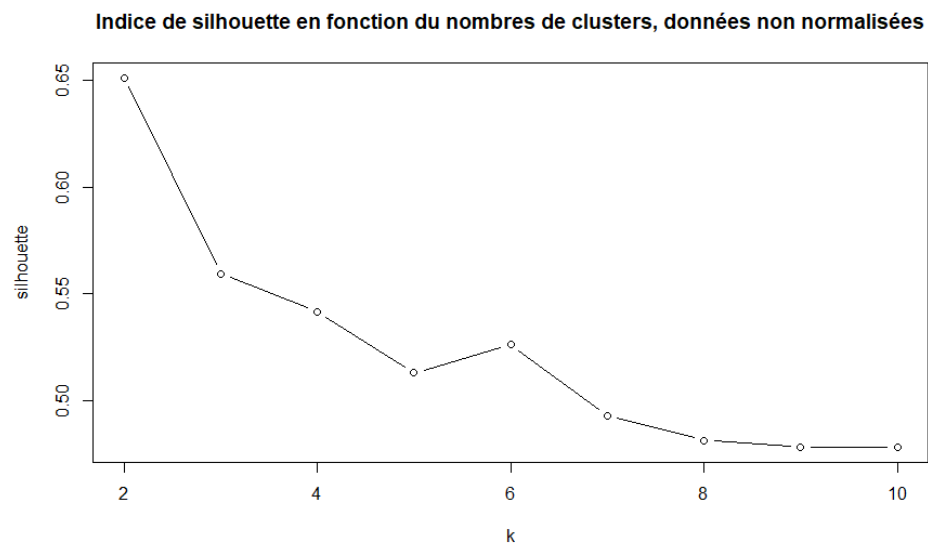
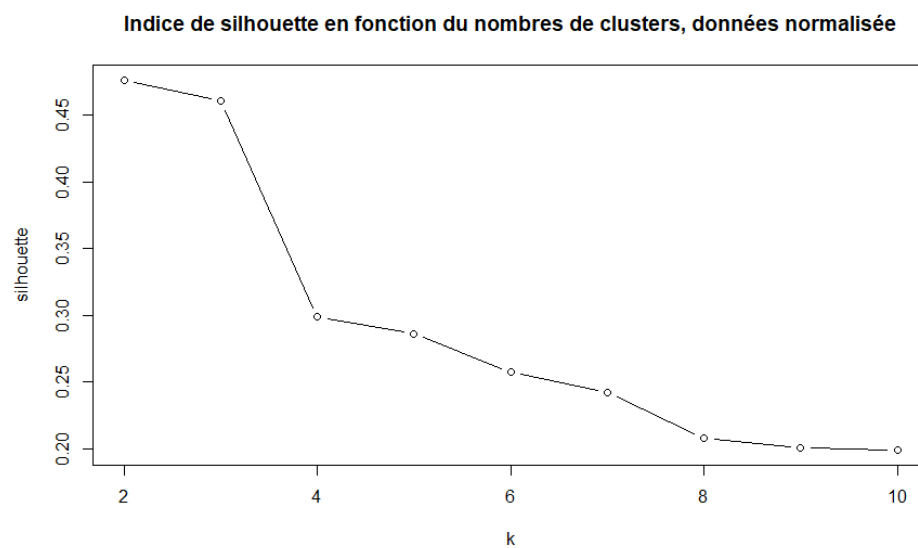


FIGURE 8 – Indice de silhouette en fonction du nombre de cluster pour jeu de données normalisées (haut) et non normalisée (bas)

Malgré une différence d'échelle, la conclusion est identique : on conserve deux clusters. Afin de comparer nos résultats aux données d'origine, on utilise un indice de Rand. Pour une CAH à deux groupes, on trouve un indice de Rand à environ 0.013, soit environ 1% de données bien classées : c'est médiocre. Avec des données non normalisées, l'indice de Rand s'élève à environ 13%, ce n'est pas non plus concluant.

Le meilleur indice de Rand est obtenu avec $k = 5$ composantes principales pour 2 groupes et s'élève à environ 0,34, c'est mieux mais toujours décevant car loin de 1.

k	Indice de Rand
2	0.22
3	0.01
4	0.28
5	0.34
6	0.07
7	0.04

TABLE 1 – Indices de silhouette pour différentes valeurs de k dans la recombinaison $PC1 + PC2 + \dots + PCk$

Ce résultat contient évidemment du biais car on savait a priori qu'il y avait deux groupes dans le jeu de données correspondant aux deux types de raisin : Kecimen et Besni.

2 Partie II : Choisir un modèle

Pour le modèle logistique, nous avons la définition suivante : dans l'apprentissage supervisé, nous choisissons le modèle de régression logistique, à savoir :

$$\begin{aligned} \text{cases } Y_k &= \mathcal{B}(n_k, \pi(x_k)) ; k = 1, \dots, K \text{ indépendantes} \\ \text{logit}(\pi(x_k)) &= \log\left(\frac{\pi_k}{1 - \pi_k}\right) = x_k \theta \\ \mathbf{P}(Z = \text{mort} ; x_k) &= \pi(x_k) = \frac{e^{x_k \theta}}{1 + e^{x_k \theta}} \end{aligned}$$

Dans ce projet, n_k est égal à 1 puisqu'il ne s'agit que de problèmes de classification.

Avant de commencer, nous devons prétraiter les données, c'est-à-dire les normaliser. Cela permet d'éviter que les données de l'échantillon aient des valeurs trop élevées et faussent ainsi l'estimation des paramètres, en particulier : Pour la variable X_k

$$\hat{X}_k(n) = (X_k(n) - \bar{X}_k) / \sigma(X_K)$$

```
1 #centrage et reduction
2 X = scale(Raisin[, -p], center=TRUE, scale=TRUE) / sqrt((n-1) / n)
3 raisin=cbind(X, Raisin[, p])
```

Dans la pratique, il bénéficie des avantages suivants :

- **Améliore la stabilité numérique** : les algorithmes d'optimisation (comme la descente de gradient) convergent plus facilement quand les variables sont sur des échelles comparables.
- **Amélioration de la performance des modèles régularisés (Lasso, Ridge)** : la pénalisation est plus équitable entre les variables.
- **Visualisation et diagnostic plus aisés** : les résidus et l'influence des variables sont plus simples à analyser.

Ensuite, sur la base de la première partie, nous avons utilisé l'ACP pour projeter les données de test sur les deux premiers axes principaux des données d'apprentissage, facilitant ainsi l'analyse ultérieure.

```
1 test_pca_scores <- predict(appre, newdata = test_data[, -p])$
  coord[, 1:2]
```

Le résultat final(le premier axe) :

```
1  4          6          7          13          15
   17          18          20          21
2 -2.560193778 -2.119447443 -2.493345729 -2.762428418 -0.638802680
   -1.092720125 -1.307513393 -0.746716250 -1.118500687
3          29          35          36          37          39
           41          43          46          49
4 -0.933140228 -3.141534475 -3.274318113 -2.034790362 -0.478553184
   0.142099360 -0.668582829 -1.589749564 -2.261860116...
```

Nous procédons maintenant à la sélection du modèle prédictif sur la base d'indicateurs multiples.

2.1 La modélisation

Pour la sélection des hyperparamètres, dans les deux premiers modèles de régression glm, les variables explicatives sont déjà présélectionnées et il suffit d'utiliser l'algorithme d'optimisation pour sélectionner les paramètres qui maximisent la fonction de vraisemblance. Pour la sélection du discriminant AIC, en revanche, nous utilisons l'algorithme **backward**(regard anova!), afin de trouver le modèle explicatif qui présente le plus petit discriminant AIC.

Cependant, pour le modèle de régression lasso, nous devons déterminer les coefficients du terme de pénalité.

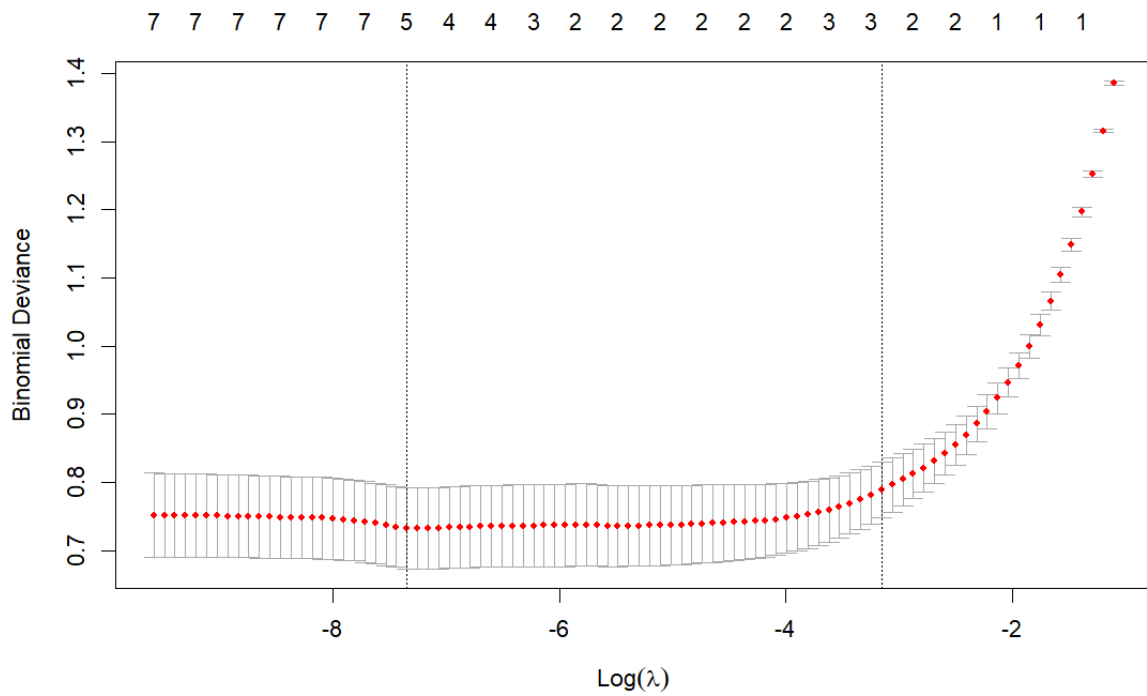
$$\hat{\theta}(\tau) = \arg \min_{\theta \in \mathbb{R}^p} \|Y - X\theta\|^2 + \tau \|\theta\|_1$$

Nous avons d'abord sélectionné le coefficient optimal à l'aide de **la validation croisée**(bloc=10 par défaut),

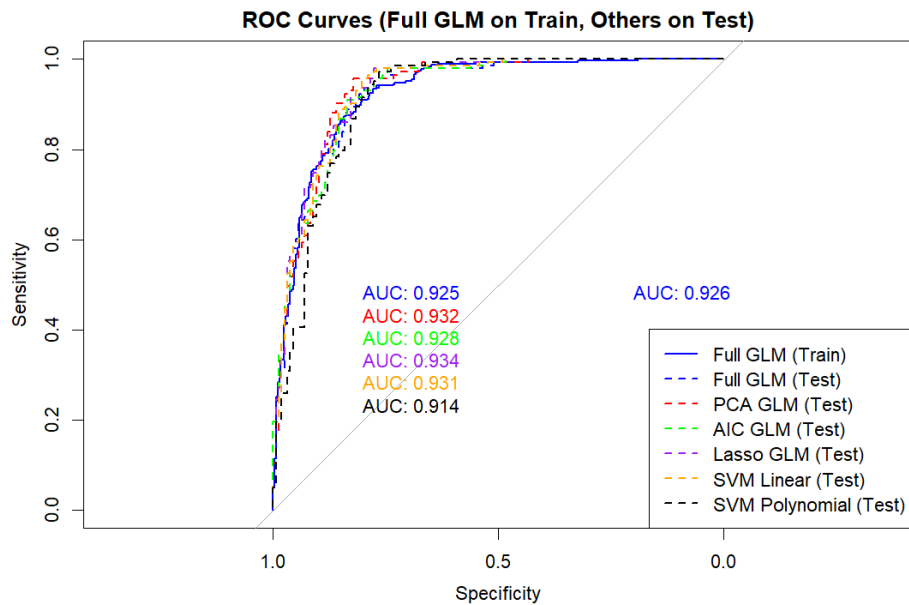
```
1 # alpha = 1 lasso
2 lasso_model <- glmnet(x_train, y_train, family = "binomial",
  alpha = 1, lambda = cv_lasso$lambda.min)
```

et après avoir fixé ce coefficient, le modèle a été utilisé à nouveau pour réaliser l'estimation des paramètres.

```
1 Optimal lambda for Lasso: 0.0006471181
```



2.2 ROC



Une attention particulière doit être accordée au tracé des courbes ROC autour du modèle SVM : La machine à vecteurs de support (SVM) produit une fonction de décision $f(x)$, qui mesure la distance entre un échantillon et l'hyperplan de séparation. Cependant, cette sortie n'est pas une probabilité.

Pour obtenir une estimation probabiliste, on applique une transformation sigmoïde appelée **Platt Scaling**, qui approxime la probabilité que l'étiquette soit positive :

$$P(y = 1 \mid x) = \frac{1}{1 + \exp(Af(x) + B)}$$

où :

- $f(x)$ est la sortie de la fonction de décision du SVM,
- A et B sont des paramètres ajustés par maximum de vraisemblance sur un ensemble de validation.

Cette méthode permet de transformer les scores bruts en probabilités calibrées, nécessaires pour des courbes ROC ou pour interpréter les prédictions.

```

1 AUC on Training Set (Full GLM only):
2 Full GLM: 0.9259152
3
4 AUC on Test Set:
5 Full GLM: 0.9250813
6 PCA GLM: 0.9318961
7 AIC GLM: 0.9277983
8 Lasso GLM: 0.9342568
9 SVM Linear: 0.9311389
10 SVM Polynomial: 0.9141686

```

2.3 Les erreurs et choix finale

Nous utilisons ici la règle de Bayes pour dériver les prédictions : Soit a_0 le coût de choisir 1 à la place de 0, et a_1 le coût de choisir 0 à la place de 1. Alors le prédicteur de Bayes :

$$h^*(x) = \mathbb{I}_{\{\pi(x) > \frac{a_0}{a_1 + a_0}\}}, \text{ où } \pi(x) = \mathbb{P}(Y = 1 \mid X = x).$$

minimise le risque $\mathbb{P}(h(x) \neq Y)$, coût moyen de l'erreur de prédiction.

Les erreurs :

Modèle	Erreur sur l'ensemble d'entraînement	Erreur sur l'ensemble de test
Régression logistique (glm)	86	44
Régression logistique + ACP (glm2)	78	36
Régression logistique avec sélection AIC	87	44
Régression Lasso	82	41
SVM (linéaire)	86	125
SVM (polynomial)	149	140

TABLE 2 – Comparaison des erreurs de prédiction pour différents modèles

En comparant les coefficients AUC et le nombre d'erreurs, nous pouvons voir que les régressions des modèles **glm** basées uniquement sur les deux premières valeurs principales et **lasso** fonctionnent le mieux.

En ce qui concerne les prédictions des modèles, nous avons constaté que pour les première et troisième données de l'ensemble d'apprentissage, tous les modèles ont prédit de manière incorrecte, car ces deux ensembles de données s'écartaient considérablement du sujet.

Deuxièmement, la meilleure performance du modèle glm basé sur l'ACP peut être due au fait que les valeurs *cos2* de toutes les variables sont grandes et que les deux premières valeurs principales ont donc bien extrait la plupart des informations, réduisant ainsi la complexité du modèle et l'interférence des fluctuations imprévisibles dans le modèle prédictif (bruit blanc).

C'est ce qui explique la supériorité du modèle lasso, avec sa fonction de filtrage des variables, qui rend l'incertitude introduite par les variables du modèle moins influente (**Area et ConvexArea**) et améliore la prédiction du modèle. **Attention ! Les seuils discriminants que nous utilisons à ce stade peuvent faire une différence dans l'efficacité du modèle.**

Quant aux modèles SVM, qu'ils soient linéaires ou à noyaux polynomiaux, ils donnent de mauvais résultats sur l'ensemble de test qui, comme on le verra à la section 3, est indivisible, de sorte que nous écartons cette partie de la classification.

3 Partie III

3.1 LDA

L'analyse discriminante linéaire (LDA) s'applique à des variables suivant une loi gaussienne de variance identique notée Σ . On suppose également que la frontière est linéaire.

Dans ce modèle, le classifieur associé à chaque classe c s'écrit pour $x \in \mathbb{R}^k$ où k est le nombre de variables : $\delta_c(x) = x'\Sigma^{-1}\mu_c - \frac{1}{2}\mu'_c\Sigma^{-1}\mu_c + \log(\pi_c)$, où $\mu_c \in \mathbb{R}^k$ est la moyenne de chacune des variables sur la classe c , et $\pi_c = \mathbb{P}(Y = c)$, où Y représente les données.

Par la suite, on note μ_B et μ_C les moyennes des variables, respectivement sur les classes Besni et Kecimen.

Dans notre cas, nous avons uniquement deux classes, pour obtenir l'équation de la frontière, on résout $\delta_B(x) = \delta_K(x)$. On a :

$$\begin{aligned}\delta_B(x) = \delta_K(x) &\iff x'\Sigma^{-1}\mu_B - \frac{1}{2}\mu'_B\Sigma^{-1}\mu_B + \log(\pi_B) = x'\Sigma^{-1}\mu_K - \frac{1}{2}\mu'_K\Sigma^{-1}\mu_K + \log(\pi_K) \\ &\iff x'\Sigma^{-1}(\mu_B - \mu_K) + \frac{1}{2}(\mu'_K\Sigma^{-1}\mu_K - \mu'_B\Sigma^{-1}\mu_B) + \log\left(\frac{\pi_B}{\pi_K}\right) = 0\end{aligned}$$

En posant $a = \Sigma^{-1}(\mu_B - \mu_K)$ et $b = \frac{1}{2}(\mu'_K\Sigma^{-1}\mu_K - \mu'_B\Sigma^{-1}\mu_B) - \log\left(\frac{\pi_B}{\pi_K}\right)$, cela se réécrit plus simplement en $x'a + b$ qui est donc l'équation de notre frontière.

De plus, on rappelle que cette section ne traite que des deux premières composantes de l'ACP normée, et qu'à ce titre on peut écrire $x = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \in \mathbb{R}^2$. Enfin, notons que les coefficients a et b font intervenir μ_C, μ_B et $\text{Var}(X_1), \text{Var}(X_2)$, que l'on estime sur les variables issues de l'ACP de la façon suivante :

$$\begin{aligned}\mu_C &= \begin{pmatrix} \frac{1}{n_C} \sum_{i \in C} X_{1,i} \\ \frac{1}{n_C} \sum_{i \in C} X_{2,i} \end{pmatrix} \\ \mu_B &= \begin{pmatrix} \frac{1}{n_B} \sum_{i \in B} X_{1,i} \\ \frac{1}{n_B} \sum_{i \in B} X_{2,i} \end{pmatrix} \\ \Sigma &= \frac{(n_B - 1)\Sigma_B + (n_C - 1)\Sigma_C}{n_B + n_C - 2}\end{aligned}$$

où $n = n_C + n_B$ est le nombre total d'observations, n_C et n_B sont respectivement le nombre d'observations de la classe Kecimen et Besni.

Après l'implémentation de l'algorithme, on obtient la figure suivante :

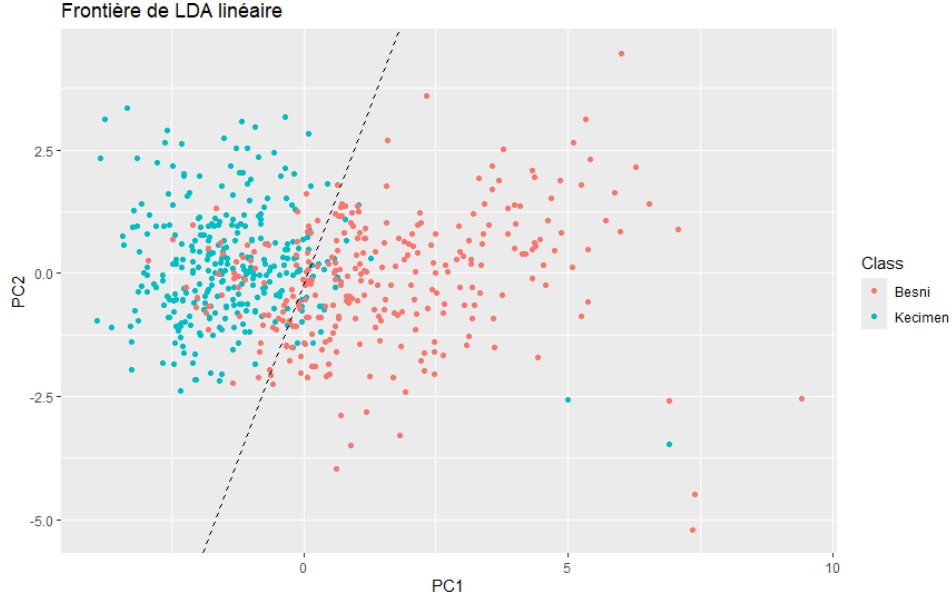


FIGURE 9 – Résultat de l’algorithme LDA

L’erreur de classification sur le jeu test, définie par la somme des faux positifs et négatifs, est de 37 sur un total de 300 observations, soit environ 12% d’erreur.

3.2 Courbe ROC pour LDA

Pour déterminer $\pi_c(x) = P(Y = c \mid X = x)$, on utilise le théorème de Bayes : $\pi_c(x) = \frac{f(x|Y = c)\mathbb{P}(Y = c)}{f(x)}$.

Or dans le cas d’une analyse linéaire discriminante, on a : $f(x) = \sum_{c=1}^C \mathbb{P}(Y = c)\varphi_c(x)$.

En injectant l’expression de la fonction caractéristique d’une gaussienne, et sachant que $\Sigma_c = \Sigma$, il vient :

$$\pi_c(\mathbf{x}) = \frac{\mathbb{P}(Y = c) \exp -\frac{1}{2} [(\mathbf{x} - \mu_c)' \Sigma^{-1} (\mathbf{x} - \mu_c)]}{\sum_{\ell} \mathbb{P}(Y = \ell) \exp -\frac{1}{2} [(\mathbf{x} - \mu_{\ell})' \Sigma^{-1} (\mathbf{x} - \mu_{\ell})]}$$

La règle de classification de Bayes consiste alors à prendre $\hat{h}(x) = \arg \min_{c \in C} \pi_c(x)$. Fort de cette remarque, on peut alors construire une nouvelle règle de classification en introduisant un seuil, qui va nous permettre de définir une courbe ROC.

Les résultats sont les suivants :

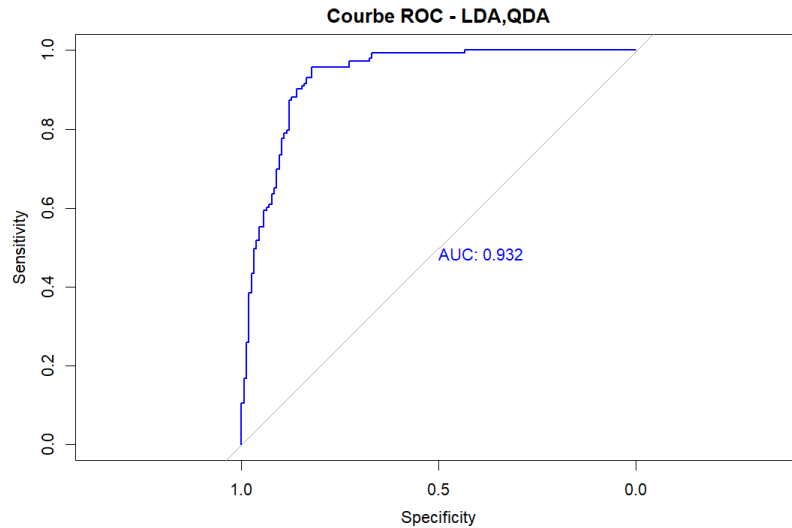
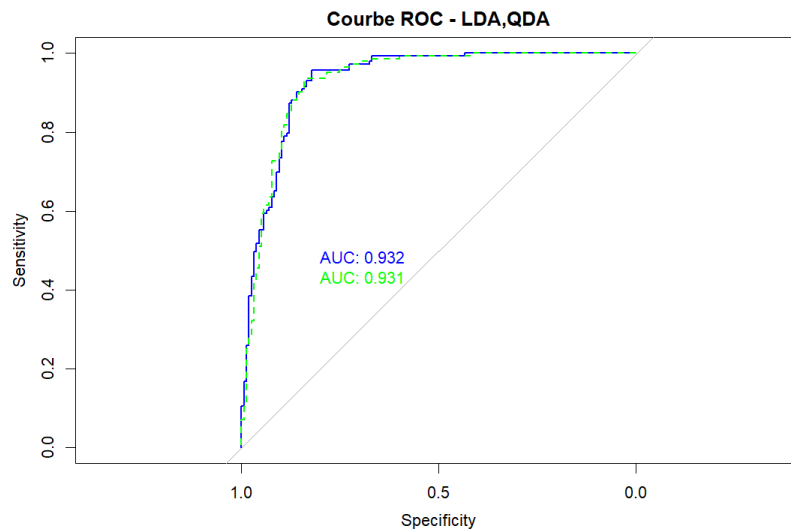


FIGURE 10 – Courbe ROC

L'AUC sous cette courbe est estimée à 0.93, ce qui est concordant avec la courbe ROC obtenue à l'aide du package pROC. Par rapport à l'analyse discriminante quadratique, les coefficients de correction et d'AUC de LDA sont significativement plus élevés.



```
1 > 1-mean(test_data_pca$Class==pred_qda_test$class)
2 [1] 0.1533333
3 > auc(roc_plot2)
4 Area under the curve: 0.9306
```

Par conséquent, l'analyse discriminante quadratique n'a pas optimisé le modèle. De même, nous pouvons considérer le modèle à toutes les variables. On a le résultat suivant :

```
1 > 1-mean(test_data$Class==pred_lda2_test$class)
2 [1] 0.1433333
3 Area under the curve: 0.9285
```

La simulation est moins bonne que le modèle précédent. Par conséquent, pour la sélection du modèle, nous nous en tenons toujours aux conclusions de la deuxième section.